

ARQUITECTURA DE COMPUTADORAS

CISC, RISC y Segmentación de Instrucciones

Teoría fundamental de procesadores, pipelining y diseño monociclo — con ejercicios resueltos paso a paso.

JJhosep Rodrigo Arocutipa Mamani 2024-119027

CISC

RISC

PIPELINE

MONOCICLO

EJERCICIOS

Contenido de la sesión



Segmentación (Pipeline)

Las 5 etapas clásicas y fórmulas de rendimiento de Stallings



Arquitectura No Segmentada

Procesador monociclo y cálculo del camino crítico



Arquitectura RISC

Filosofía load-store, formato fijo y banco de registros



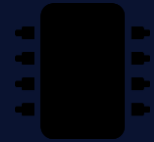
Arquitectura CISC

Filosofía, operaciones registro-memoria e instrucciones complejas



Ejercicios Resueltos

Aplicación práctica de cada modelo teórico paso a paso

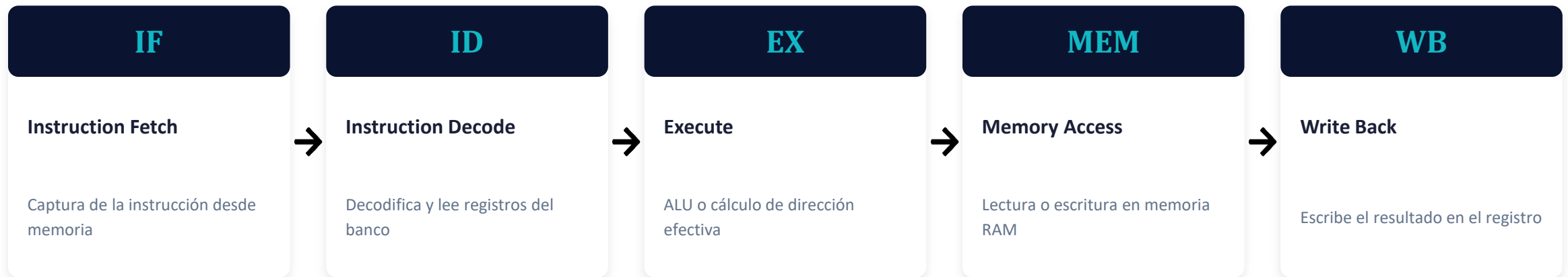


5

bloques temáticos

Segmentación de instrucciones (Pipeline)

El pipelining solapa la ejecución de múltiples instrucciones, como una línea de ensamblaje, incrementando el rendimiento sin alterar la frecuencia del reloj.



Cada etapa toma 1 ciclo de reloj. Con instrucciones ideales sin riesgos, el pipeline permite iniciar una instrucción nueva cada ciclo, en lugar de esperar a que termine la anterior por completo.

Fórmulas de rendimiento (Stallings)



Ciclos totales (ideal)

$$\text{Ciclos} = k + (n - 1)$$

k = etapas, n = instrucciones



Tiempo total segmentado

$$T_k = [k + (n - 1)] \times \tau$$

τ = ciclo de reloj



Aceleración (Speedup)

$$S_k = (n \times k) / (k + n - 1)$$

Comparado con procesador mono-etapa



Riesgos de control: penalización por saltos

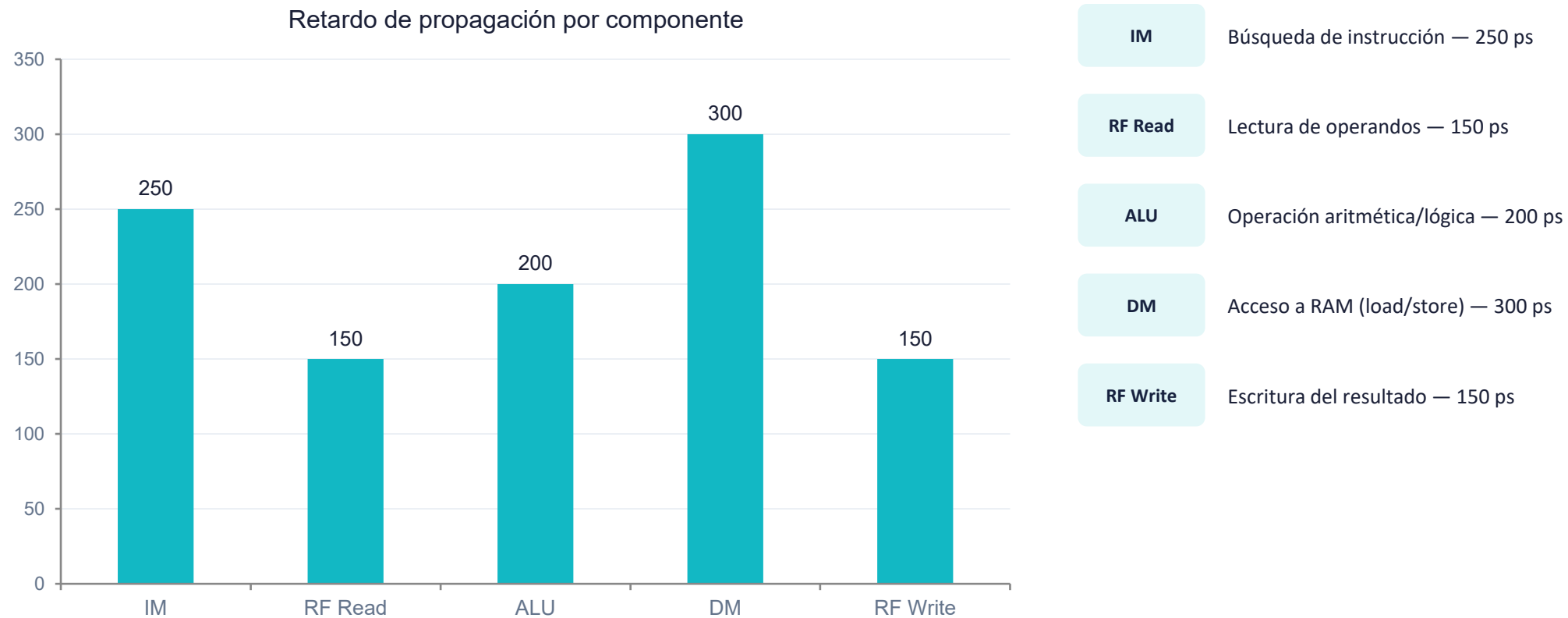
Cuando un salto se resuelve tarde, las instrucciones especulativas ya cargadas deben descartarse (flush), añadiendo ciclos extra al total ideal.

$$\text{Ciclos} = k + (n - 1) + [n \cdot p \cdot q \cdot (k - 1)]$$

p = prob. de salto · q = prob. de que se tome

Procesador no segmentado (monociclo)

Toda instrucción inicia, se procesa y finaliza dentro de un único ciclo de reloj. El periodo del reloj debe ser tan largo como la instrucción más lenta del set.



Camino crítico y periodo del reloj

Tipo-R (add)

IM 250 + RF 150 + ALU 200 + RFW 150

750 ps

lw (carga)

IM 250 + RF 150 + ALU 200 + DM 300 + RFW 150

1050 ps

beq (salto)

IM 250 + RF 150 + ALU 200

600 ps

$T = \text{MAX}(750, 1050, 600)$

1050 ps

Periodo de reloj del sistema

$f = 1 / T \approx 952.38 \text{ MHz}$

Frecuencia máxima de operación



Ineficiencia del diseño monociclo

En un beq, la operación termina a los 600 ps pero el ciclo dura 1050 ps: 450 ps se desperdician (eficiencia 57.1%). Esta pérdida motivó la migración hacia arquitecturas segmentadas.

Filosofía RISC: Reduced Instruction Set Computer

La arquitectura RISC se fundamenta en la simplicidad del hardware: instrucciones simples y uniformes que se ejecutan idealmente en un solo ciclo de reloj por etapa.



Filosofía Load-Store

Solo las instrucciones LOAD y STORE acceden a memoria RAM; el resto opera sobre registros.



Formato fijo de instrucción

Simplifica la decodificación en el pipeline y regulariza el tiempo de cada etapa.



Banco de registros amplio

Más registros internos reducen los accesos repetitivos a memoria.

SECUENCIA RISC TÍPICA

```
LW  R1, 0(R0)
```

```
LW  R2, 4(R0)
```

```
ADD R5, R1, R2
```

Cada instrucción hace una sola cosa: cargar, sumar o guardar — nunca todo a la vez.

Operandos

Solo registros

Formato

Longitud fija

Objetivo

Pipeline eficiente

Filosofía CISC: Complex Instruction Set Computer

La arquitectura CISC minimiza la cantidad de instrucciones de un programa mediante instrucciones robustas y multifuncionales, grabadas directamente en el microcódigo del hardware.



Énfasis en el hardware

El microcódigo interno ejecuta operaciones complejas en una sola instrucción.



Menos instrucciones por programa

Cada instrucción realiza varias tareas: buscar dato, calcular y guardar.



Apoyo directo al compilador

Instrucciones que se asemejan a las estructuras de lenguajes de alto nivel.

INSTRUCCIÓN CISC TÍPICA

```
ADD AX, [0x104]
```

Una sola línea: busca el operando en memoria RAM y ejecuta la suma en el mismo ciclo lógico de instrucción.

Operandos

Registro ↔ Memoria

Longitud

1 a 15+ bytes (x86)

Objetivo

Código compacto

Características clave de CISC



Operaciones Registro-Memoria

Las instrucciones aritméticas toman operandos directamente de la RAM, ejecutando captura y cómputo en una sola instrucción.



Instrucciones de longitud variable

Pueden medir desde 1 byte hasta más de 15 bytes (x86), optimizando almacenamiento pero complicando la decodificación.



Énfasis en el software

Se facilita el trabajo del compilador con comandos complejos similares a lenguajes de alto nivel.

CISC vs. RISC: comparación directa

Criterio	CISC	RISC
Acceso a memoria	Directo desde instrucciones aritméticas	Solo mediante LOAD / STORE
Longitud de instrucción	Variable (1 a 15+ bytes)	Fija y uniforme
Nº de instrucciones/programa	Menor (código compacto)	Mayor (código regular)
Complejidad del hardware	Alta (microcódigo)	Baja (control cableado)
Ciclos por instrucción	Variable, frecuentemente > 1	Idealmente 1 por etapa
Ejemplo de ejercicio	6 instrucciones	9 instrucciones



EJERCICIOS RESUELTOS

Aplicación paso a paso de CISC, RISC, Pipeline y Monociclo

Resultado = $(A + B) \times (C - D)$ en CISC

Dirección	Variable	Valor
0x100	A	12
0x104	B	8
0x108	C	15
0x10C	D	10
0x110	Resultado	→ 100

CÓDIGO ENSAMBLADOR (x86)

```

MOV AX, [0x100]    ; AX = A (12)

ADD AX, [0x104]    ; AX = 12+8 = 20

MOV BX, [0x108]    ; BX = C (15)

SUB BX, [0x10C]    ; BX = 15-10 = 5

MUL BX             ; AX = 20*5 = 100

MOV [0x110], AX    ; Resultado = 100

```



6 instrucciones totales — el acceso registro-memoria permite resolver toda la expresión sin instrucciones LOAD independientes.

Resultado = $(A + B) \times (C - D)$ en RISC

CÓDIGO ENSAMBLADOR RISC

```
LI    R0, 0x100
LW    R1, 0(R0)    ; A=12
LW    R2, 4(R0)    ; B=8
LW    R3, 8(R0)    ; C=15
LW    R4, 12(R0)   ; D=10
ADD   R5, R1, R2   ; 20
SUB   R6, R3, R4   ; 5
MUL   R7, R5, R6   ; 100
SW    R7, 16(R0)
```



Inicialización

R0 = dirección base 0x100



Load

4 LW cargan A, B, C, D a registros



ALU pura

ADD, SUB y MUL solo sobre registros



Store

SW escribe el resultado (100) en RAM

9 instrucciones

vs. 6 en CISC — más líneas, pero cada una es uniforme y lista para el pipeline.

4 instrucciones independientes en pipeline de 5 etapas

Instr.	C1	C2	C3	C4	C5	C6	C7	C8
I1 (ADD)	IF	ID	EX	MEM	WB	-	-	-
I2 (SUB)	-	IF	ID	EX	MEM	WB	-	-
I3 (AND)	-	-	IF	ID	EX	MEM	WB	-
I4 (OR)	-	-	-	IF	ID	EX	MEM	WB

Sin riesgos ni dependencias, el pipeline se llena en el ciclo 5 y se vacía en los ciclos 6 a 8.

Ciclos = $k + (n-1)$

= 5 + 3 = 8 ciclos

Frente a 20 ciclos en modo secuencial

Speedup

2.5x

20 ciclos / 8 ciclos = 2.5

Ejercicios numéricos de segmentación

Ejercicio 1

Tiempo total

$k=5, n=100, \tau=2 \text{ ns}$

$$T_k = 104 \times 2 = 208 \text{ ns}$$

Ejercicio 2

Speedup

$T_1 = 100 \times 5 \times 2 = 1000 \text{ ns}$

$$S_k = 1000/208 \approx 4.81$$

Ejercicio 3

Etapas desbalanceadas

$\tau = \max(1.2, 1.5, 1.0, 1.1) + 0.2$

$$\tau = 1.7 \text{ ns} \rightarrow T_k = 22.1 \text{ ns}$$

Ejercicio 4

Penalización por salto

Flush de 2 ciclos, $n=20$

$$\text{Ciclos} = 5 + 19 \times 2 = 26$$



Ejercicio 5 — CPI efectivo real

Con CPI ideal = 1, 20% de saltos (penalización 2 ciclos) y 15% de dependencias de datos (retraso 1 ciclo):

$$CPI_{\text{efectivo}} = 1 + (0.20 \times 2) + (0.15 \times 1) = 1.55$$

1.55 ciclos por instrucción

Eficiencia del ciclo en un salto condicional (beq)



Tiempo real beq

600 ps

IM+RF+ALU

Tiempo desperdiciado

450 ps

$1050 - 600$

Eficiencia

57.1%

$(600/1050) \times 100$

Esta pérdida sistemática en instrucciones cortas —forzadas a esperar el ciclo completo— es la razón histórica de la migración hacia arquitecturas segmentadas (pipeline).

CONCLUSIÓN

Cuatro modelos, un mismo objetivo: velocidad



CISC

Prioriza código compacto delegando complejidad al hardware.



RISC

Prioriza simplicidad para maximizar la eficiencia del pipeline.



Pipeline

Solapa etapas y multiplica el rendimiento (hasta 2.5x en el ejemplo).



Monociclo

Modelo simple, pero ineficiente: desperdicia hasta 43% del ciclo.